DevOps

The Second Way - The Principles of Feedback

predavanja su bazirana i islike preuzete iz knjige
“The **DevOps Handbook**: How to Create World-Class Agility, Reliability, & Security in Technology Organizations”
Gene Kim, Jez Humble, Patrick Debois, John Willis

Principi povratnih informacija

- Dok prvi pristup, principi toka, opisuje tehnike koje treba koristiti kako bi se poboljšao protok informacija i rada od razvojnih timova ka operativnim, ovaj drugi pristup opisuje principe i tehnike koji su potrebni kako bi se obezbedio brz i konstantan protok povratnih informacija (od operativnog tima ka razvojnom) - tok sdesna ulevo u svim razvojnim fazama

- Ovo je posebno važno kada se radi o kompleksnim sistemima, kod kojih bi, bez uvođenja ovakvih principa, prva prilika da se uoči i ispravi greška bila tek kada je neki katastrofalni otkaz već u toku.

Principi povratnih informacija

- U tehnološkoj sferi rad se obično obavlja u složenom sistemu - vrlo često se ključne greške otkrivaju tek kada se desi neki masovni otkaz funkcionalnosti, kod koga se ključne funkcionalnosti gube u produkcionom okruženju - direktno nanoseći štetu klijentu (sigurnosni problemi, krađa ličnih podataka iz sistema…)

- Kako bi se ovo izbeglo, neophodno je razvijati sigurne i pouzdane softverske sisteme, a to se postiže obezbeđivanjem dobrog protoka svih informacija kroz ceo tok proizvodnje - kako unapred ka narednim fazama, tako i u povratnom smeru.

- Ovo omogućava da se problemi otkriju u ranim fazama, dok su još mali, jeftiniji i jednostavniji za ispravku, i sprečimo njihovo ispoljavanje u katastofalnom obliku

Rad u velikim sistemima

- Jedna od ključnih karakteristika da nijedna osoba nema kompletan uvid u celokupan sistem i način kako se svi detalji komponuju u kompleksan sistem

- Vrlo često ovakvi sistemi su zasnovani na uzajamnom dejstvu mnoštva komponenti, a funkcionisanje sistema nije prosti zbir pojedničanih funkcionalnosti komponenti, već je rezultat njihove interakcije, koja može biti različita od slučaja do slučaja.

- Otkazi u velikim sistemima su u nekom momentu neminovni.

- Druga karakteristika je da ponavljanje istovetnog rada, istih koraka, ne mora u svim slučajevima dovesti do istog ishoda (ishod nije unapred predvidiv) - zbog ovoga čisto praćenje statički definisanih “checklist-i” nije dovoljno da se izbegnu katastrofalni otkazi

Stvaranje sigurnog radnog okruženja u velikim sistemima

- Kad su već otkazi pre ili kasnije neminovni treba stvoriti siguran sistem za rad (bez obzira da li se radi o proizvodnji ili visokotehnološkom sektoru kao što je IT), radno okruženje u kome svako može svoj rad obavljati rasterećen znajući da ako se greška i desi, postoji dobro uhodan proces koji će tu grešku otkriti brzo, mnogo pre nego što ona izazove veće probleme.

Stvaranje sigurnog radnog okruženja u velikim sistemima

- Dr. Steven Spear - koji je analizirao uzročno posledične (kauzalnost) mehanizme u Toyota Product System - došao je do postulata da je dizajn savršeno sigurnih sistema verovatno nedostižno, ali ih možemo učiniti maksimalno sigurnim u okvirima naših mogućnosti ako se poštuju sledeći uslovi:
  - Složenim radnim zadacima se upravlja na takav način da se pravovremeno otkriju problemi u dizajnu i razvoju proizvoda
  - Problemi se grupišu i rešavaju, što dovodi do brzog usvajanja novog znanja
  - Ovako nastalo novo (lokalno) znanje se brzo širi i koristi u svim delovima organizacije
  - Vođe konstantno stvaraju nove vođe koji kontinuirano napreduju u određenim oblastima

Pravovremeno identifikovanje problema

- Kako bi se kreiralo sigurno i pouzdano radno okruženje koje donosi visokokvalitetne rezultate, neophodno je konstantno analizirati i testirati sva predložena rešenja (dizajn) i sve pretpostavke kako će se nešto ponašati u realnim uslovima (operativna upotreba).

- Što više početnih pretpostavki dokažemo da nisu validne to ćemo brže otkloniti potencijalne probleme koje bi daljnji rad pod tim pretpostavkama neminovno uneo u krajnji rezultat.

- Ključni mehanizam za ovo je uspostavljanje povratnih petlji, ali i prosleđivanja informacija unapred u razvojnom okruženju.

Pravovremeno identifikovanje problema

- U visokoefikasnim radnim okruženjima postoji dobro definisan protok informacija kroz ceo razvojni tok - svaka radna operacija se meri i nadzire, svaki defekti ili znatno odstupanje od specifikacije se lako otkriva i rešava.

- U tehnološkoj sferi baš kada nema brzih povratnih informacija dobijaju se nekvalitetna softverska rešenja.

  - Primer: u “metodi vodopada” može doći do situacije da se kod razvija godinu dana bez ikakve povratne informacije, koje počinju da pristižu tek kada se krene u fazu intenzivnog testiranja “gotovog” proizvoda, ili još gore tek kada se isporuči klijentu. U ovoj situaciji povratne informacije su prespore i nedovoljne da bi na vreme uticale na tok razvoja.

Pravovremeno identifikovanje problema

- U efikasnom razvojnom procesu kreiraju se pouzdani, jasni i brzi tokovi informacija kako unapred, ka sledećim fazama u razvojnom ciklusu, tako i tokovi povratnih informacija, koji obuhvataju sve: Product Management, Development, QA, Information Security, Operations
    - ovo se postiže tako što svi u lancu koriste odgovarajuće alate da prate stanje razvoja, a sve faze se automatizuju u maksimalno mogućoj meri, tako da to uključuje automatizovano bildovanje, integraciju, test procese. Svaki od ovih alata će automatski prijaviti svaku promenu koja izaziva grešku u bilo kojoj od ovih faza.
    - neophodno je i kreirati odgovarajuće merljive indikatore (telemetrija) kako bi se lakše utvrdilo koje komponente ne dostižu postavljene standarde i ciljeve

Pravovremeno identifikovanje problema

- Povratni tok informacija ne samo do omogućava brzo otkrivanje i oporavak od grešaka, već obezbeđuju i vredne informacije kako izbeći da se problem ponovi.

“Kada sam vodila odeljenje kvaliteta, svoj posao sam opisivala kao kreiranje povratnih ciklusa. Povratne veze su kritične, jer omogućavaju upravljanje. Moramo konstantno validirati rezultate u odnosu na korisnička očekivanja, naš željeni rezultat (namere) i našu stvarnu realizaciju”

Elisabeth Hendrickson,
VP of Engineering, Pivotal Software

Grupisanje problema i okupljanje radi njihovog rešavanja

- Naravno samo detektovanje problema nije dovoljno - kada se problem desi neophodno je okupiti tim koji je neophodan za rešavanje uočenog problema

- Okuljanje (swarming - rojenje) interventnog tima i analiza problema treba da spreči njihovo širenje i ponavljanje. Tim koji analizira i rešava problem stvara novo, svaki put sve detaljnije, znanje o celokupnom sistemu.

Grupisanje problema i okupljanje radi njihovog rešavanja

- Evidentni predstavnik ovog pristupa je tzv. *Andon-cord* u Toyotinom proizvodnom lancu, koje svaki zaposleni (bez obzira na poziciju) može povući kada god vidi da nešto nije u redu (nema dela, rad traje duže nego je predviđeno…), vođa tima se tako alarmira da postoji problem u proizvodnoj liniji, i pokušava odmah da ga reši, ako ne uspe uzadatom vremenu, proizvodna linija se automatski zaustavlja - svi se okupljaju kako bi zajednički rešili problem.

- Na ovaj način izbegava se pristup “popravićemo kad bude više vremena” koji obično rezultuje sa nikad ili tek kad nešto zaista krene naopako.

- Korišćenjem ovog principa svi se okupljaju kako bi doprineli svojim idejama u rešavanju problema

Grupisanje problema i okupljanje radi njihovog rešavanja

- Okupljanje i rešavanje problema je neophodno jer otklanja:
  - propagaciju greške duž proizvodnog lanca, čime bi i cena same greške i cena njenog naknadnog uklanjanja biti samo veća
  - sprečava da se započne rad na novom zadatku koji bi u datim okolnostima verovatno samo uneo nove greške u sistem
  - ukoliko se problem ne reši, na istom mestu bi se greška verovatno uskoro ponovila

Grupisanje problema i okupljanje radi njihovog rešavanja

- Ova praksa je kada je uvedena bila u kontradiktornosti sa ustaljenim praksama upravljanja, jer se dozvoljava da lokalni problem poremeti i ostatak organizacije.

- Ipak ovaj princip zapravo omogućava da se pravovremeno steknu nove, vredne informacije o tome kako greška nastaje, i kako je izbeći, i na taj način sprečava se “gubitak informacija”

  - Ovo je posebno bitno u složenim sistemima gde uzrok greške može biti neka jedinstvena kombinacija interakcija koja će se možda nekada opet ponoviti

Grupisanje problema i okupljanje radi njihovog rešavanja

- U visokotehnološkim radnim procesima mora se napraviti virtuelni analog “Andon cord-a”

- Neophodno je osim toga stvoriti i kulturu rada koja podstiče bilo kog pojedinca koji uoči grešku da ukaže na nju i zaustavi (ako je to potrebno) dalji razvoj dok se problem ne reši. Npr. bilo ko uoči da je neko pre njega ili on sam uneo izmenu koja “ruši” kontinualni build ili test proces, treba odmah to i da signalizira.

- Kada se desi ovaj prekid, svi se angažuju na rešavanju problema, i istovremeno se pazi da se nikakav dodatni rad ne uvede u sistem dok se problem ne reši. (Na ovaj način se obezbeđuje da kontinualna integracija i *deployment* funkcionišu i dalje).

Konstantno promovisanje kvaliteta na samom izvoru

- Nesigurne principe rada možemo i slučajno promovisati na osnovu načina kako reagujemo na incidente. U složenim sistemima prosto dodavanje novih koraka provere i odobrenja može rezultovati povećanjem verovatnoće da se greška ponovi.

- Problem je što sama efikasnost procesa odobravanja opada što je on dalji od mesta na kome se stvarni rad (razvoj) dešava.

- U takvim situacijama ne samo da su konačne odluke bazirane na lošijim kriterijumima, već se i ceo ciklus razvoja usporava.

- Usporeni ciklusi, dodatno, slabe povratne informacije - jer je njihova relevantnost manja kako vreme protiče.

Konstantno promovisanje kvaliteta na samom izvoru

- Kada kod kontrolisanih “top-down” sistema dođe do problema obično se ispostavlja da je došlo do razmimoilaženja između “ko treba da uradi nešto” i “ko zaista nešto radi”.

- Primeri neefikasne kontrole kvaliteta:
  - angažovanje drugog tima da završi teške i poslove sklone greška i manualne zadatke koji bi mogli biti automatizovani
  - očekivanje odobrenja od prezauzetih osoba koje su često odsutne - faktički one donose odluke bez dovoljno podataka i u žurbi
  - kreiranje velike količine dokumentacije problematičog nivoa detaljnosti i tačnosti, koja postane zastarela vrlo brzo
  - prosleđivanje velikih blokova zadataka u razvoj i naknadno odobravanje i dugotrajno čekanje da se to obavi

Konstantno promovisanje kvaliteta na samom izvoru

- Pravilan pristup:
  - Svaki član tima treba da se brine za kontrolu kvaliteta i ispravljanje grešaka u delu nad kojim ima uvid i kontrolu. Na ovaj način se kontrola kvaliteta i donošenje odluka o tome približava mestu gde se stvarni rad i obavio.
  - Koristi se koncept odobravanja od strane timskih kolega kako bi se potvrdilo da unesene i predložene promene rade kako je predviđeno (i ne narušavaju prethodne funkcionalnosti)
  - Automatizuje se koliko je god moguće provera kvaliteta koju obavljaju QA i Info security. Na taj način umesto da razvojni tim mora da “traži” da se organizuju testovi, automatizovani testovi se mogu izvršavati “po potrebi”, tako da razvojni tim može brzo i vrlo često da testira novi kod.
  - Na ovaj način kvalitet postaje nešto o čemu se brinu svi tokom svake faze razvoja, a ne zadatak nekog izolovanog, nedostupnog tima.

Omogućavanje optimizacije za naredne korake u lancu razvoja

- *Lean practices* razlikuju dve vrste klijenata prema kojima se treba “dizajnirati” i realizovati rešenje (ono što razvijate)
  - Eksterni klijent - koji plaća za isporučene usluge i proizvode (pa i softver)
  - Interni klijent - sledeća karika u lancu razvoja koja preuzima dosad urađeno u sledećem koraku
- optimizacija pretpostavlja da direktnog internog klijenta - onog koji od nas prezuima posao u sledećem koraku - posmatramo kao najbitnijeg

Omogućavanje optimizacije za naredne korake u lancu razvoja

- Optimizovanje našeg posla treba da ima na umu i sve probleme koje možemo prouzročiti u sledećem koraku razvoja, a što bi automatski usporilo protok informacija i rada ka konačnom završetku.

- U tehnološkom procesu razvoja - optimizaciju za naredni korak u lancu radimo tako što razvijamo “za operativni tim”, pri čemu se vodi računa da operativni nefunkcionalni zahtevi (arhitektura, performanse, stabilnost, mogućnost testiranja, konfigurabilnost rešenja i sigurnost) moraju imati istu važnost kao i funkcionalni korisnički zahtevi.

- Na ovaj način garantuje se da kvalitet rešenja razvija već na samom početku, vrlo često i u formi preporučenih nefunkcionalnih zahteva koje težimo da ispunimo u svakom rešenju koje se razvija.